# VolTune: A Fine-Grained Runtime Voltage Control Architecture for FPGA Systems

Akram BEN AHMED, Takahiro HIROFUCHI, Takaaki FUKAI
*Intelligent Platforms Research Institute*
*National Institute of Advanced Industrial Sciences and Technology*
2-3-26 Aomi, Koto-ku, Tokyo, JAPAN
{akram.benahmed,t.hirofuchi,takaaki.fukai}@aist.go.jp

***Abstract*—The rapid emergence of edge computing platforms and large-scale data centers has made power efficiency a primary design constraint, particularly for data-intensive and AI-driven workloads. Field-programmable gate arrays (FPGAs) are increasingly adopted due to their flexibility and potential for energy-efficient acceleration. However, FPGA supply voltages are typically fixed at design time using conservative margins, limiting the ability to adapt power consumption to runtime conditions.**

**This paper presents VolTune, an open-source runtime voltage control architecture that enables runtime tuning of FPGA supply voltages through FPGA-integrated control logic that abstracts low-level PMBus operations. VolTune provides both hardware-based and software-based control paths, allowing designers to balance deterministic low-latency operation against programmability.**

**In the presented prototype, the hardware-based control path achieves a measured end-to-end voltage transition latency of 2.3 ms, while the controller adds under 2% static power overhead and under 2% FPGA resource overhead. As a representative case study, VolTune is evaluated on the GTX transceiver supply rail of a Kintex-7 platform. The results show that runtime voltage tuning exposes a bounded operating region with clear trade-offs between energy efficiency and reliability, and achieves up to approximately 29.3% rail-power reduction at 10.0 Gbps when allowing BER up to $10^{-6}$. These results show that FPGA-integrated runtime voltage control can provide practical energy savings with low integration overhead.**

***Index Terms*—FPGA, runtime voltage control, PMBus, serial transceivers, energy efficiency**

## I. Introduction

The rapid growth of edge computing platforms and large-scale data centers has placed power efficiency at the center of system design [1], [2]. Emerging applications such as data analytics, machine learning, and real-time signal processing demand high performance under strict energy and thermal constraints, particularly in distributed edge-cloud environments where power and cooling budgets are inherently limited [2]. As a result, architectural flexibility and energy-efficient execution have become critical requirements across a wide range of computing environments.

Field-programmable gate arrays (FPGAs) have gained significant traction in these domains due to their reconfigurability, high throughput, and favorable performance-per-watt characteristics compared to general-purpose processors [3]. Major technology companies have made sustained investments in FPGA-based infrastructure for both data center and edge deployments, integrating FPGAs into production systems for acceleration, networking, and system offload, and positioning them as a key building block for future edge-cloud architectures [4]–[6].

Despite this growing role, power management in FPGA-based systems has not evolved at the same pace as architectural capabilities. In current practice, FPGA supply voltages are typically fixed at design time using conservative margins to guarantee correct operation across worst-case process, voltage, and temperature conditions. Commercial FPGA platforms such as Xilinx and Intel devices specify nominal core and transceiver voltages with tight operating ranges, leaving limited opportunity for runtime adaptation within standard design flows [7], [8]. While this approach simplifies validation, it prevents systems from adapting power consumption to runtime conditions, workload variation, or application-level tolerance.

In practice, workload characteristics in edge and data center systems vary significantly over time, including changes in computation intensity, memory access patterns, and communication activity [9]. During low-utilization or communication-light phases, operating all FPGA voltage rails at worst-case margins results in unnecessary dynamic and static power consumption. Prior studies have demonstrated that supply voltage reduction can yield quadratic reductions in dynamic power, with measured FPGA power savings exceeding 20–40% under controlled undervolting scenarios [10]. Moreover, many applications, such as machine learning inference, multimedia processing, and approximate data analytics, exhibit inherent tolerance to bounded errors or minor performance variations, enabling operation at reduced voltage without catastrophic failure [11], [12]. Flexible voltage control provides an architectural mechanism that can be leveraged to exploit temporal slack and application-level tolerance in pursuit of improved energy efficiency.

Although modern FPGA platforms provide programmable voltage regulators accessible through standardized interfaces such as PMBus [13], these interfaces are typically exposed at the board-management or software level and require low-level command sequences. The resulting control latency and integration complexity make frequent or fine-grained runtime voltage adjustments difficult to integrate within standard FPGA design flows. Consequently, runtime voltage tuning remains rarely

This work is partially supported by JSPS KAKENHI Grant Number ZBA2029720A.

integrated as a first-class architectural mechanism in FPGA-based systems.

This paper addresses this gap by proposing *VolTune*, a runtime voltage control architecture that integrates voltage control directly into FPGA logic. VolTune abstracts low-level PMBus operations behind a structured control interface, enabling voltage adjustment to be triggered at runtime through hardware or software control paths. Unlike prior undervolting approaches, VolTune is explicitly designed for runtime use, supports fine-grained voltage adjustment within regulator resolution limits, and characterizes control overhead and transition behavior.

The contributions of this work are summarized as follows:

- An FPGA-integrated voltage-control architecture for runtime supply tuning, including a structured abstraction of PMBus-based regulation for hardware and software control paths.
- A detailed characterization of the VolTune controller, including control latency, voltage transition behavior, and hardware overhead.
- An experimental evaluation of runtime voltage tuning on the transceiver supply rail of a Kintex-7 platform.
- To support reproducibility and follow-on use, the VolTune implementation is released as an open-source repository containing the design artifacts used in this work [14].

The remainder of this paper is organized as follows. Section II reviews the background and related work. Section III presents the VolTune architecture and system model. Section IV describes the PMBus control engine implementation and associated design trade-offs. Section V evaluates the VolTune controller in terms of latency and overhead. Section VI presents a representative case study using serial transceivers. Section VII discusses limitations and generality. Finally, Section VIII concludes the paper.

## II. Technical Background and Related Work

### A. Power Management in FPGA-Based Systems

Power management in FPGA-based systems has historically relied on static, board-level configuration, rather than closed-loop, runtime adaptation. A central difficulty is that safe operating points depend on the user design's timing, as well as temperature and voltage droop, which are not known with sufficient precision at design time.

*Chow et al.* [15] presented one of the early demonstrations that dynamic voltage scaling can be applied to commercial FPGAs without modifying the silicon, by using an on-chip delay sensor (LDMC) and a closed-loop controller that adjusts the FPGA supply voltage to track timing margin across temperature changes. Their setup uses external voltage control, and the key contribution is the runtime feedback mechanism that links measured delay to voltage selection, rather than a fixed guardband.

*Levine et al.* [16] moved closer to practical DVFS for user designs by proposing online slack measurement insertion (SMI) and using the measured slack to drive dynamic voltage and frequency scaling decisions. This line of work explicitly targets timing monitoring during operation, and highlights that DVFS control is constrained by what can be instrumented in the fabric, with overheads in logic and clocking resources.

*Farhadi et al.* [17] demonstrated that implementing PMBus access in programmable logic reduces latency and overhead compared to software execution on the processing system. Their work focuses on monitoring accuracy and control-path comparison. In contrast, VolTune formalizes an FPGA-integrated runtime voltage-control architecture that exposes a structured command abstraction and supports systematic controller characterization and workload-level evaluation.

From a power-delivery and calibration perspective, *Zhao et al.* [18] proposed a self-calibrating DVFS scheme that characterizes voltage and frequency operating limits across temperature points, stores them in a table, and then applies DVFS at runtime according to measured temperature. A key practical aspect is that their method explicitly interacts with a digitally controlled DC-DC converter, and treats converter response and quantization as part of the DVFS design, rather than assuming idealized voltage transitions.

### B. Undervolting and Timing Speculation

Reducing supply voltage below nominal levels is a well-established technique for lowering dynamic and static power consumption, but it introduces timing violations as circuit delays increase. To mitigate this, architectural mechanisms for detecting and recovering from timing errors have been proposed.

*Ernst et al.* [19] introduced Razor, a circuit-level timing speculation mechanism that augments flip-flops with shadow latches to detect late-arriving signals. When a timing error is detected, the pipeline recovers using architectural replay. This work demonstrated that systems can operate beyond conservative voltage margins by dynamically detecting and correcting timing failures, thereby enabling energy savings without permanent functional errors.

Extending this idea, *Das et al.* [20] proposed enhanced in-situ error detection techniques that improve robustness under aggressive voltage scaling. Their work further established that error-aware voltage scaling can be made practical through microarchitectural support.

In the FPGA domain, undervolting behavior has been studied experimentally rather than through architectural error recovery. *Salami et al.* [10] performed a comprehensive evaluation of supply voltage underscaling in FPGA on-chip memories. They characterized failure modes, error rates, and the relationship between voltage reduction and memory reliability, demonstrating measurable power savings at reduced voltage levels while quantifying the onset of errors. In follow-up work, the same authors evaluated the built-in Error-Correction Code (ECC) in FPGA on-chip memories under aggressive undervolting [21], showing that the SECDED ECC mechanism can correct over 90% of undervolting faults in BRAMs and enable further power savings with minimal application accuracy loss when used with neural network accelerators. They later extended their study to reduced-voltage operation across multiple on-chip FPGA components in the context of CNN acceleration [22], highlighting significant energy-efficiency

gains and characterizing the corresponding reliability behavior across different blocks within the FPGA fabric.

*Ben Ahmed et al.* [23] presented a preliminary exploration of undervolting for error-permissive serial interconnection in multi-FPGA systems, using KC705 transceivers as a case study. That work examined the relationship between transceiver supply voltage, bit error rate, and power reduction under approximate communication settings, and identified runtime voltage adaptation as a promising direction. However, it did not implement an FPGA-integrated voltage-control architecture, did not abstract PMBus access into a reusable control interface, and did not characterize the latency and overhead of the control path itself. VolTune extends that direction by introducing a concrete runtime voltage-control mechanism that can be invoked from FPGA logic and evaluated independently from the application case study.

Collectively, these empirical studies establish that undervolting can yield significant energy savings in FPGAs, and that fault behavior under aggressive undervolting is characterizable and, in some cases, mitigable. However, these works primarily focus on characterization and mitigation techniques, rather than providing an architectural abstraction that enables systematic runtime voltage control within FPGA logic.

### C. Approximate Computing and Error Tolerance

Approximate computing is based on the observation that many applications can tolerate bounded inaccuracies in exchange for improved energy efficiency. Although many FPGA studies focus on arithmetic datapaths and accelerator kernels, the same broader principle is relevant to communication and system-level behavior when bounded errors can be tolerated without unacceptable degradation in application outcomes.

*Han and Orshansky* [11] introduced the concept as a cross-layer design paradigm that relaxes strict correctness to reduce energy. *Mittal* [12] surveyed architectural and circuit-level techniques that trade accuracy for improvements in power, delay, or area, and organized the design space of approximation methods.

These ideas are particularly relevant to FPGA accelerators, because FPGAs allow designers to customize arithmetic datapaths and numeric representations in hardware. *Ullah et al.* [24] proposed FPGA-oriented approximate multipliers that exploit LUT structures and carry chains, and reported gains in area, latency, and energy relative to vendor multiplier IPs, while bounding accuracy loss. *Perri et al.* [25] designed approximate adders specialized for FPGA LUT resources by approximating the least significant bits, and evaluated error metrics and implementation costs on an Artix-7 device.

Approximation has also been applied at the accelerator level for machine learning. *Korol et al.* [26] proposed a configurable framework that swaps accurate and approximate multipliers in FPGA CNN accelerators to navigate the accuracy-performance-resource trade-off at the edge.

Overall, these FPGA-focused studies support the practical premise that controlled approximation can be implemented in FPGA fabrics at the arithmetic and accelerator levels. More broadly, they support the view that bounded-error operation can be a useful design point when application-level behavior remains acceptable. This motivates runtime mechanisms that can exploit application-level tolerance while keeping system behavior within acceptable bounds.

### D. Positioning and Architectural Gap

While prior work has demonstrated dynamic voltage scaling using delay monitoring, architectural timing speculation, and empirical undervolting characterization, these approaches do not provide a reusable FPGA-integrated control abstraction for structured runtime rail tuning. Similarly, prior FPGA approximation studies mainly focus on arithmetic units and accelerator-level design choices, rather than runtime control of board-level supply rails for communication-oriented operating points. What remains insufficiently explored is a reusable architectural abstraction that integrates board-level voltage regulation directly into FPGA logic in a structured and runtime-invocable manner. VolTune addresses this gap by embedding a structured voltage-control interface within the FPGA platform, abstracting low-level PMBus transactions and enabling systematic controller characterization and workload-level evaluation under dynamic voltage conditions.

## III. VolTune Architecture

### A. Design Objectives and Scope

VolTune is designed to expose board-level voltage control as a runtime architectural mechanism within FPGA systems. Rather than relying on external firmware or manual configuration, it integrates voltage-control functionality into the FPGA fabric through a structured interface that can be coordinated with application execution and system state. The design is guided by the following principles:

- **Runtime Invocation:** Voltage transitions must be triggerable during system operation, without requiring reconfiguration of the FPGA bitstream or manual board-level intervention. The architecture therefore provides a command-driven interface that can be invoked programmatically at runtime through either hardware or software control paths.
- **Rail-Level Granularity:** VolTune operates at the level of discrete supply rails exposed by the board's programmable regulator. It does not modify the internal silicon power grid or attempt fine-grained cell-level control. Voltage adjustments are constrained to the resolution and limits supported by the external regulator.
- **FPGA-Resident Control Path:** VolTune places the voltage-control functionality inside the FPGA platform rather than relying on an external host-side control loop. Depending on the implementation, command sequencing can be realized either directly in FPGA logic or through an embedded processor subsystem, allowing different trade-offs between determinism and flexibility.
- **Abstraction of Low-Level Transactions:** Board-level voltage regulators are commonly accessed through PMBus command sequences. VolTune abstracts these low-level protocol transactions behind a structured control

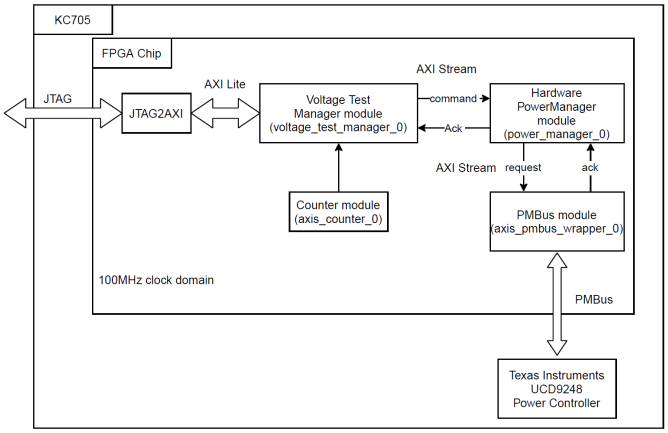



Fig. 1: VolTune hardware implementation block diagram

interface, allowing higher-level logic or software to request voltage changes without directly managing PMBus signaling details.

- **Explicit Characterization of Transition Behavior:** Voltage scaling is treated as a measurable and characterizable process. The architecture accounts for command transmission latency, regulator response time, and settling behavior, rather than assuming instantaneous voltage changes.

The scope of VolTune is intentionally limited. It does not implement circuit-level timing speculation or error-recovery mechanisms. It does not modify the FPGA silicon or internal voltage distribution network. It does not assume application-level fault tolerance. Instead, VolTune provides a reusable architectural layer that exposes controlled, runtime voltage adjustment within the operational limits specified by the FPGA vendor and regulator hardware. This scoped design allows VolTune to remain portable across platforms that provide digitally programmable regulators, while preserving compatibility with standard FPGA tool flows.

Within VolTune, the core voltage-control functionality is implemented as a PowerManager subsystem, with either a hardware or software realization. Both realizations expose a common control model, while their implementation-specific trade-offs are discussed in the following subsections.

### B. VolTune Hardware Implementation Overview

Figure 1 shows the block diagram of the hardware realization of VolTune on the KC705 platform. The realization includes the core VolTune control path together with auxiliary modules used for configuration and evaluation in the prototype implementation.

***Voltage Test Manager Module:*** The Voltage Test Manager module represents the application side of the system and can host the main FPGA workload, such as AI inference or transmission-related processing. It interfaces with the PowerManager subsystem whenever runtime voltage adjustments are required.

In the prototype implementation shown in Figure 1, the same module is also used to orchestrate controlled voltage-transition experiments by issuing voltage-change commands to the Hardware PowerManager through an AXI Stream interface. For configuration and evaluation, external register access is provided through the JTAG2AXI path shown in the figure, while a Counter module in the same clock domain is used to support repeatable timing measurements during latency characterization.

***Hardware PowerManager Module:*** The Hardware PowerManager module performs the main control operations inside the FPGA fabric. It receives command messages from the Voltage Test Manager module through the AXI Stream interface. These commands request actions such as voltage updates and telemetry readback.

Upon receiving a command, the Hardware PowerManager module translates the request into a sequence of operations directed to the PMBus module. It also receives acknowledgment signals from the PMBus module and forwards the corresponding status information back to the Voltage Test Manager module.

***PMBus Module:*** The PMBus module serves as the low-level transaction engine between the Hardware PowerManager and the external programmable power controller. It executes PMBus transactions over the SCL and SDA lines and returns acknowledgment or readback information to the Hardware PowerManager. In the prototype platform, this interface connects to a Texas Instruments UCD9248 multi-rail programmable power controller [27], which applies the requested voltage updates to the selected board-level supply rail. The internal interface and transaction behavior are detailed in Section IV.

### C. VolTune Software Implementation Overview

VolTune also includes a software realization of the PowerManager subsystem, referred to as the Software PowerManager. In this design, the PowerManager logic executes on a MicroBlaze soft processor while maintaining the same high-level command semantics as the hardware implementation.

Figure 2 shows the complete software implementation architecture as captured from the AMD Xilinx Vivado design environment, while Figure 3 provides a simplified block-diagram view of the same organization. In this realization, a MicroBlaze-based control subsystem issues voltage-control requests in software and drives the same PMBus-controlled regulator used in the hardware implementation. These requests are translated into PMBus transactions issued to the external Texas Instruments UCD9248 Power Controller [27] over the PMBus interface [13].

As shown in Figure 2, the MicroBlaze processor is connected to memory and peripheral modules through an AXI-based interconnect. PMBus access is realized through AXI-connected peripherals, enabling the software to issue the required transaction sequences and receive acknowledgment status.

### D. Design Trade-offs

The hardware realization favors deterministic sequencing and a more direct control path, which is useful when voltage control must be tightly coupled with runtime FPGA activity.

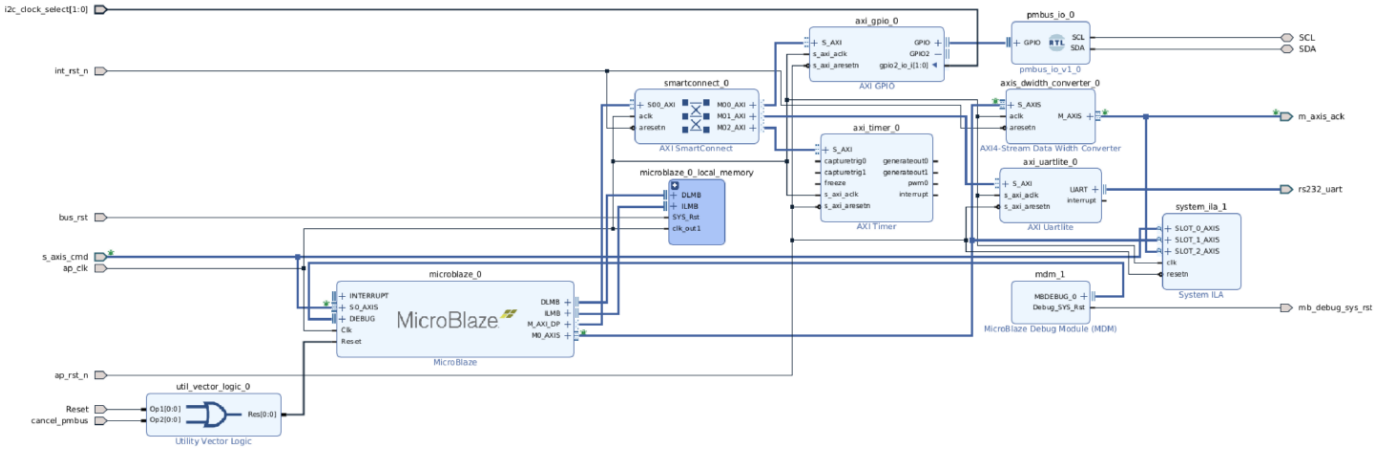



Fig. 2: Software implementation block design snapshot from Vivado

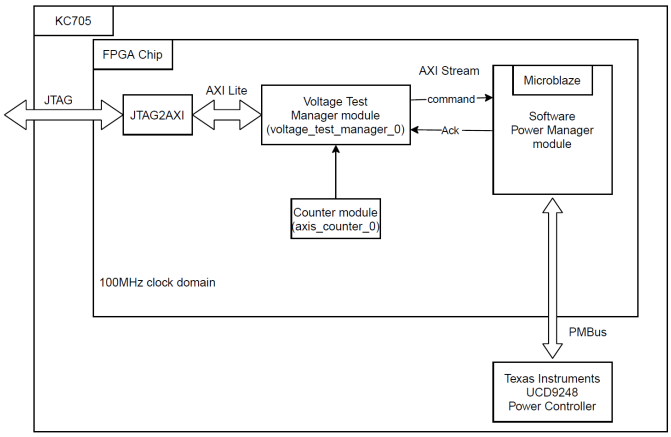
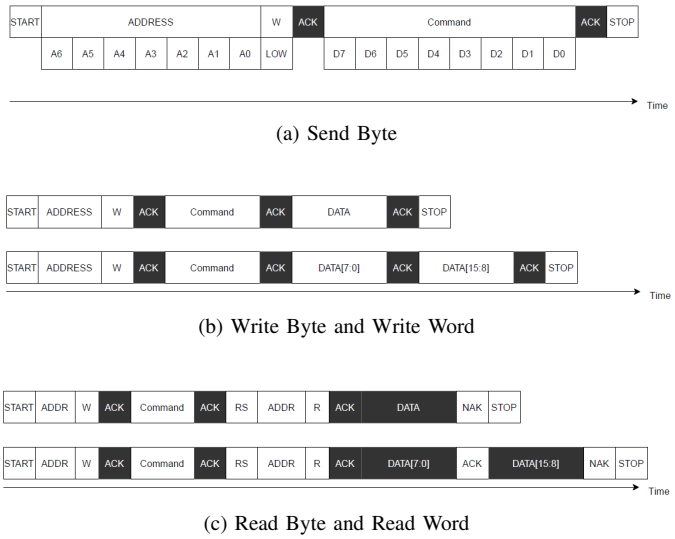



Fig. 3: VolTune software implementation block diagram




(a) Send Byte

(b) Write Byte and Write Word

(c) Read Byte and Read Word

Fig. 4: PMBus transaction primitives.

The software realization favors flexibility and ease of modification, but requires additional infrastructure, including a processor subsystem, memory, and AXI-connected peripherals, which increases resource usage and can add software and subsystem latency to the control path. In both cases, overall voltage-transition behavior remains constrained by serialized PMBus transactions and external regulator settling time, which motivates the controller characterization presented in the next section.

## IV. PMBus Control Engine Implementation

This section describes how VolTune controls KC705 voltage rails through the TI UCD9248 power controller [27] using PMBus [13]. As introduced in Section III, the PMBus module serves as the low-level transaction engine between the Hardware PowerManager and the external power controller. Here, we focus on the transaction primitives, the request conversion path, and the concrete command sequences used for voltage programming and readback on KC705.

For simplicity, VolTune implements only the subset of PMBus commands required by the target workflows, rather than the full PMBus command language. In this section, the PMBus control engine and command translation logic are implemented in RTL and HLS, so additional PMBus commands can be added by extending the command translation layer when needed. To avoid confusion between control layers, we refer to the internal command identifiers exchanged between Voltage Test Manager and PowerManager as VolTune opcodes, while the standardized PMBus command bytes transmitted to the UCD9248 are referred to as PMBus commands.

### A. PMBus Basics

PMBus is a power-management protocol that uses an $I^2C$-compatible two-wire bus. Communication occurs on SCL and SDA, where each transaction is framed by a START condition and a STOP condition, as summarized in Figure 4. The START condition is defined by SDA transitioning from 1 to 0 while SCL is high, and the STOP condition is defined by SDA transitioning from 0 to 1 while SCL is high. A transaction begins with a 7-bit device address followed by a single R/W bit, and each transferred byte is followed by an acknowledgment bit on the ninth clock pulse [13].

TABLE I: Subset of PMBus commands used by VolTune.

| Code | Command name | R/W | Typical use |
|---|---|---|---|
| 00h | PAGE | R/W | Select target rail (multi-rail devices) |
| 03h | CLEAR_FAULTS | W | Clear latched fault status |
| 21h | VOUT_COMMAND | R/W | Program output voltage setpoint |
| 43h | VOUT_UV_WARN_LIMIT | R/W | Set undervoltage warning threshold |
| 44h | VOUT_UV_FAULT_LIMIT | R/W | Set undervoltage fault threshold |
| 5Eh | POWER_GOOD_ON | R/W | Set power-good on threshold |
| 5Fh | POWER_GOOD_OFF | R/W | Set power-good off threshold |
| 8Bh | READ_VOUT | R | Read back output voltage |
| 8Ch | READ_IOUT | R | Read output current telemetry |

PMBus operations are identified by an 8-bit PMBus command. For write operations, the master sends the PMBus command followed by one or more data bytes, as shown in Figure 4b. For read operations, the master first writes the PMBus command, then performs a read transaction to receive one or more data bytes [13], as shown in Figure 4c. PMBus defines a large standardized command set, so Table I lists the subset of PMBus commands used by VolTune in the current prototype, while the complete command-language definition is provided in the PMBus Part II specification [28].

Many PMBus controllers, including multi-rail devices such as the UCD9248, multiplex multiple outputs behind a shared bus interface. Rail selection is performed using the PAGE mechanism, where the PAGE command sets an internal selector so that subsequent PMBus commands apply to the chosen rail. In practice, the controller issues PAGE when the target rail changes, then applies voltage-programming or telemetry commands for that rail [13].

### B. PMBus Module Interface in VolTune

As introduced in Section III, the PMBus module serves as the low-level transaction engine between the Hardware PowerManager and the external Texas Instruments UCD9248 Power Controller. In VolTune, each PMBus command, together with any associated payload bytes, is packaged into a stream-level request sent from the PowerManager to the PMBus module. The module then generates the corresponding bus-level signaling on SCL and SDA, including START and STOP conditions and byte-level ACK/NACK handshaking. For read operations, the module returns the received data bytes and completion status through its response channel, allowing the PowerManager to interpret the result and propagate it back to the requester. Protocol failures detected during execution are reported through structured status signals, enabling deterministic error handling without exposing bus timing details to higher-level logic.

In the present implementation, the PMBus module is operated at two clock rates, 100 kHz and 400 kHz. These PMBus clock settings are important design choices because they directly affect transaction time and therefore influence control latency and measurement granularity. Their impact on controller behavior is evaluated later in Section V.

PMBus payload values are encoded using fixed-point formats. In this design, voltage programming and readback use LINEAR16, while telemetry may use formats such as LINEAR11, depending on the PMBus command and device behavior [13], [27].

TABLE II: KC705 rail mapping.

| Lane | Rail name | PMBus addr. | PAGE |
|---|---|---|---|
| 0 | VCCINT | 52 | 0 |
| 1 | VCCAUX | 52 | 1 |
| 2 | VCC3V3 | 52 | 2 |
| 3 | VADF | 52 | 3 |
| 4 | VCC2V5 | 53 | 0 |
| 5 | VCC1V5 | 53 | 1 |
| 6 | MGTAVCC | 53 | 2 |
| 7 | MGTAVTT | 53 | 3 |
| 8 | ACCAUX_IO | 54 | 0 |
| 9 | VCCBRAM | 54 | 1 |
| 10 | MGTVCCAUX | 54 | 2 |

### C. Rail Selection on KC705

VolTune treats each supply rail as a selectable target identified by a lane number. The lane number is a VolTune-specific identifier used by the PowerManager and is not part of the PMBus standard. For each request, the PowerManager translates the lane number into a PMBus device address and a PMBus PAGE value, which together select the corresponding regulator output channel.

Table II lists the mapping used in our KC705 implementation. During operation, the controller issues a PAGE update when the target lane changes, then applies subsequent PMBus commands to the selected rail. The same approach can be adopted on other PMBus-controlled multi-rail FPGA platforms by providing the corresponding lane-to-(address,PAGE) mapping for the target board and regulator configuration.

### D. Command Packing and Conversion Path

At the application side, VolTune accepts structured requests in the form of VolTune opcodes together with a target lane and an associated value. In the prototype workflow, these requests are generated by the test infrastructure and delivered to the PowerManager through the Voltage Test Manager, as illustrated in Figure 5. The PowerManager acts as the translation layer between control software and the PMBus bus, converting each VolTune opcode request into one or more PMBus commands issued to the UCD9248. The conversion path follows three steps:

1) The PowerManager resolves the target lane into a PMBus device address and a PAGE value using the mapping in Table II.

TABLE III: VolTune opcode to PMBus command mapping.

| VolTune opcode | Operation requested from PowerManager | PMBus command(s) issued to UCD9248 |
|---|---|---|
| 0x0 | Clear Status | Internal state clear — (no PMBus transaction, controller-internal reset) |
| 0x1 | Set Under Voltage | `PAGE` (0x00, when lane changes)<br>`VOUT_UV_WARN_LIMIT` (0x43)<br>`VOUT_UV_FAULT_LIMIT` (0x44) |
| 0x2 | Set Power Good On Voltage | `POWER_GOOD_ON` (0x5E) |
| 0x3 | Set Power Good Off Voltage | `POWER_GOOD_OFF` (0x5F) |
| 0x4 | Set Voltage | `VOUT_COMMAND` (0x21) |
| 0x5 | Get Voltage | `READ_VOUT` (0x8B) |

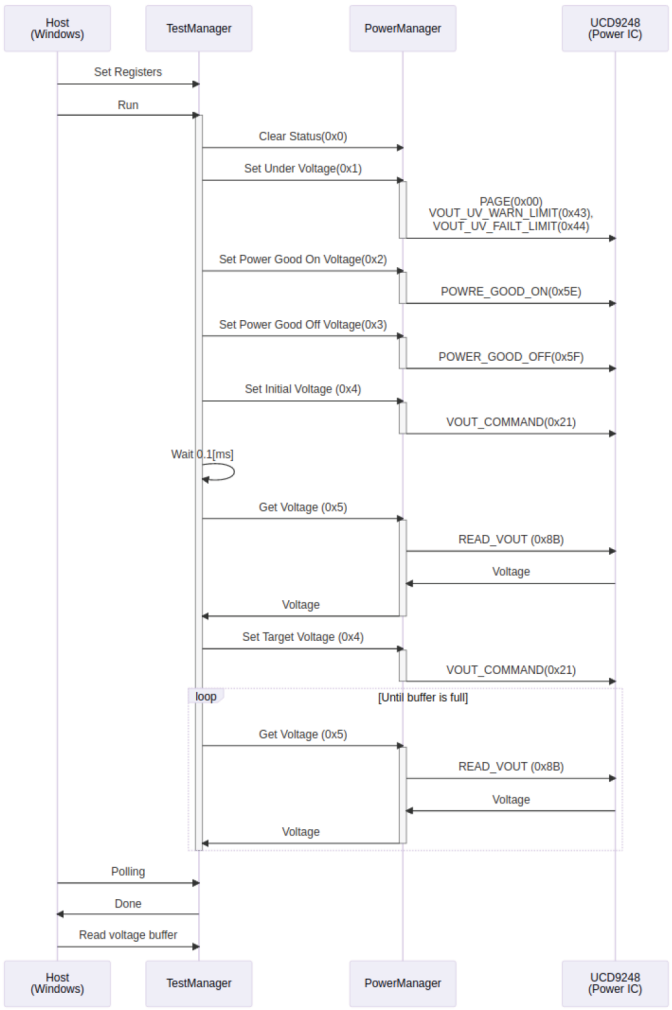



Fig. 5: Prototype voltage measurement sequence.

2) It selects the appropriate PMBus transaction type, for example Write Word for voltage and threshold programming or Read Word for voltage and telemetry readback, consistent with the PMBus primitives introduced in Section IV-A.
3) It packs the selected PMBus command and its payload bytes into the PMBus module request stream. Payload values are encoded using the fixed-point formats described earlier, for example `LINEAR16` for voltage programming and readback and `LINEAR11` for some telemetry commands.

Each PMBus transaction is executed atomically. The next transaction is issued only after the PMBus module reports completion through its response channel and status signals. This serialized execution model avoids overlapping bus activity and ensures deterministic command sequencing. As illustrated in Figure 5, a VolTune opcode request to set a rail voltage expands into an ordered PMBus sequence, including a `PAGE` update when the lane changes, followed by a Write Word transaction that issues the `VOUT_COMMAND` PMBus command with a `LINEAR16`-encoded voltage payload.

### *E. Voltage Update and Readback Sequences*

This subsection describes the concrete PMBus sequences used for voltage programming and readback. These sequences are implemented using the primitives in Figure 4 and the lane-to-(address,`PAGE`) mapping introduced in Section IV-C.

***Voltage update sequence:*** In VolTune, voltage programming is realized as an explicit PMBus sequence rather than a single abstract control action. In the prototype measurement workflow shown in Figure 5, this sequence includes threshold-register configuration followed by the final voltage-setpoint update. As a concrete example, consider a request to set the voltage of `VCCBRAM` to 0.9 V. From Table II, `VCCBRAM` corresponds to lane 9, which maps to PMBus address 54 and `PAGE`=1. Using the PMBus primitives in Figure 4 and the opcode-to-command mapping in Table III, the resulting PMBus sequence is:

1) Write Byte:
`[Addr=54][PAGE (00h)][01h]`
2) Write Word:
`[Addr=54][VOUT_UV_WARN_LIMIT (43h)][Threshold]`
3) Write Word:
`[Addr=54][VOUT_UV_FAULT_LIMIT (44h)][Threshold]`
4) Write Word:
`[Addr=54][POWER_GOOD_ON (5Eh)][Threshold]`
5) Write Word:
`[Addr=54][POWER_GOOD_OFF (5Fh)][Threshold]`
6) Write Word:
`[Addr=54][VOUT_COMMAND (21h)][LINEAR16(0.9V)]`

In this example, the PMBus address selects the regulator device and the PAGE command selects the target rail within

TABLE IV: VolTune operation mapping on KC705.

| Operation | PMBus command sequence (conceptual) | Primitive types |
|---|---|---|
| Select rail | Write `PAGE` when lane changes | Write Byte |
| Voltage update | Prototype workflow: threshold writes, then `VOUT_COMMAND` | Write Byte + Write Word (multiple) |
| Voltage readback | `PAGE` when lane changes, then `READ_VOUT` | Write Byte + Read Word |
| Telemetry readback | `PAGE` when lane changes, then periodic telemetry reads (`READ_IOUT`) | Write Byte + Read Word (periodic) |
| Controller reset | Internal state clear only, no PMBus transaction | — |

that device. The payloads denoted by `[Threshold]` represent PMBus-encoded voltage threshold values written to the under-voltage warning, under-voltage fault, and power-good registers. In the prototype workflow, these threshold values are explicitly programmed before the final `VOUT_COMMAND` update so that the regulator state remains consistent with the requested rail setting. Their exact values are chosen according to the intended operating point and the corresponding protection or monitoring limits of the target rail. This sequence reflects the prototype measurement workflow, while protocol handling remains internal to the PMBus module.

***Voltage readback sequence:*** Voltage readback is performed using a Read Word transaction with `READ_VOUT`, consistent with Figure 4c. The PowerManager waits for transaction completion at the module layer, then forwards the returned value back to the requester. The same mechanism is used for both single readback and repeated sampling.

As a concrete example, consider a request to read back the voltage of `MGTAVCC`. From Table II, `MGTAVCC` corresponds to lane 6, which maps to PMBus address 53 and `PAGE`=2. The resulting PMBus sequence is:

1) Write Byte:
   `[Addr=53][PAGE (00h)][02h]`
2) Read Word:
   `[Addr=53][READ_VOUT (8Bh)]`

Here, the PMBus address selects the regulator device and the PAGE command selects the target rail. The returned value is encoded in `LINEAR16` format and propagated back by the PowerManager as the measured rail voltage.

***How the UCD9248 interprets voltage setpoints:*** The UCD9248 does not apply `VOUT_COMMAND` directly to the DAC. As shown in Figure 6, the programmed value is selected through the device control path, then combined with calibration offset, constrained by limits, and scaled before driving the DAC reference. This internal processing reinforces the point already made in Section III: voltage adjustment must be treated as a regulator-level operation with finite response and settling time, not as an instantaneous rail change.

### *F. Mapping Summary and Timing Discipline*

Table IV summarizes how the main VolTune operations are implemented on KC705 using PMBus sequences and the transaction primitives introduced in Section IV-A. As discussed in Sections IV-C and IV-E, porting the same control approach to another PMBus-controlled multi-rail FPGA platform requires only the corresponding lane-to-(address,`PAGE`) mapping and the appropriate command sequence for each operation.

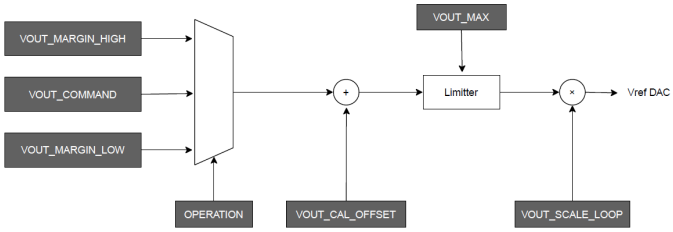



Fig. 6: UCD9248 voltage adjustment path.

VolTune enforces serialized execution of PMBus transactions. A new PMBus request is not issued until the previous request completes at the module layer and returns completion status to the PowerManager. This avoids overlapping PMBus transactions and keeps regulator state unambiguous.

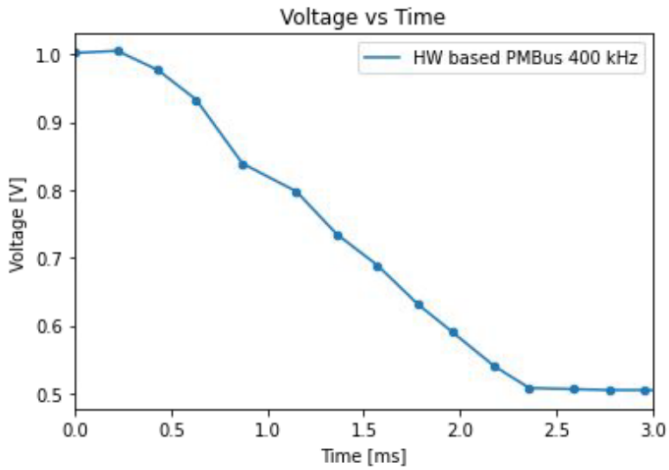
The end-to-end timing of a voltage operation is therefore determined by the transaction time on the PMBus wires together with regulator response and settling behavior. These timing components are quantified later in Section V.

## V. Controller Characterization

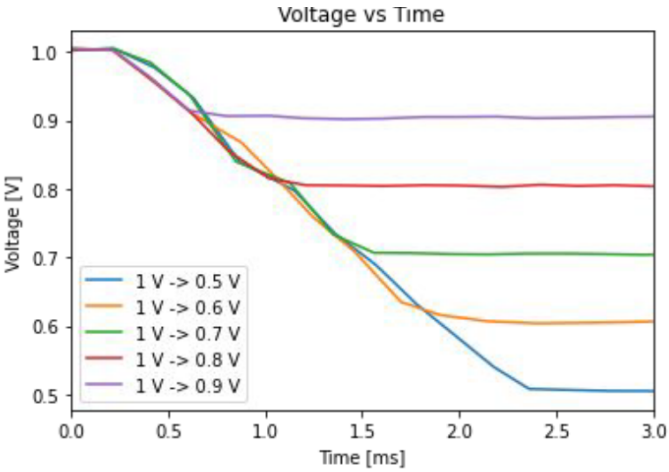
In this section, we characterize the VolTune controller as a runtime voltage-control mechanism. We focus on voltage transition dynamics, control latency, settling behavior, and the impact of control-path choices and PMBus clocking. We also report controller overhead, including FPGA resource usage and static power, to quantify the cost of integrating runtime voltage control into the system. This controller-level evaluation is presented independently from the representative case study discussed later.

### *A. Experimental Setup and Metrics*

The controller characterization is performed on a KC705-based FPGA platform equipped with a TI UCD9248 digitally programmable power controller accessed through PMBus [13], [27]. We evaluate both VolTune control paths, the hardware control path implemented in FPGA logic and the software control path implemented on MicroBlaze, while keeping the same PMBus commands and rail-selection mechanism described in Section IV. As introduced in Section IV-B, the PMBus module is operated at two clock rates, 100 kHz and 400 kHz, which are evaluated here as implementation-level design choices affecting transaction timing. Unless stated otherwise, experiments target one supply rail at a time and apply controlled voltage transitions using VolTune opcodes translated into PMBus commands by the PowerManager.




(a) 1.0 V → 0.5 V transition (HW PMBus, 400 kHz)




(b) Voltage transitions for multiple target voltages (HW PMBus, 400 kHz)

Fig. 7: Measured voltage transition dynamics using the hardware PMBus control path at 400 kHz.

Table V summarizes the settings used in this section. We sweep voltage transitions over multiple target voltages, evaluate both rising and falling transitions, and compare hardware versus software PMBus control paths under the two PMBus clock rates. This setup isolates controller behavior from application effects and enables direct comparison of transition dynamics and control overhead across configurations.

The characterization uses four metrics:

- Voltage transition latency: the elapsed time from issuing a voltage-update request at the PowerManager interface to the point at which the measured rail voltage reaches and remains within a stable region around the target.
- Settling behavior: reported using voltage-versus-time traces and derived settling times under both rising and falling transitions.
- Control-path impact: evaluated by comparing hardware versus software execution and by varying the PMBus clock rate, which changes transaction overhead on SCL and SDA.
- Controller overhead: quantified using FPGA resource utilization and static power for both implementations.

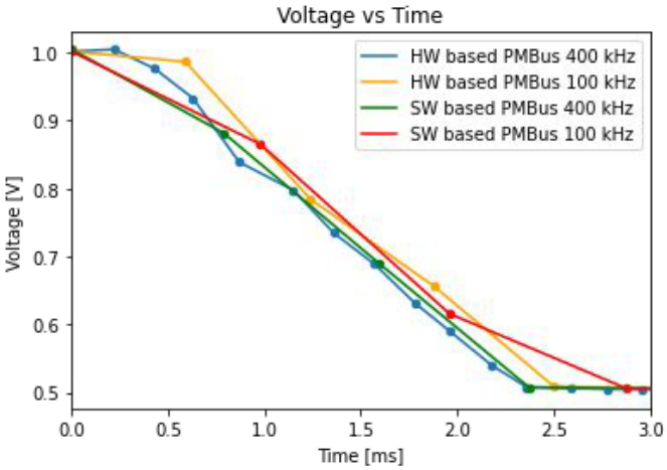



Fig. 8: Voltage transition comparison across control paths and PMBus clock rates.

TABLE V: Controller characterization test conditions.

| Item | Values |
|---|---|
| PMBus control path | Hardware-based PMBus<br>Software-based PMBus |
| PMBus clock rate | 400 kHz, 100 kHz |
| Voltage decrease sweep | 1.0 V → 0.9, 0.8, 0.7, 0.6, 0.5 V |
| Voltage increase sweep | 0.5, 0.6, 0.7, 0.8, 0.9 V → 1.0 V |

### B. Voltage Transition Latency and Dynamics

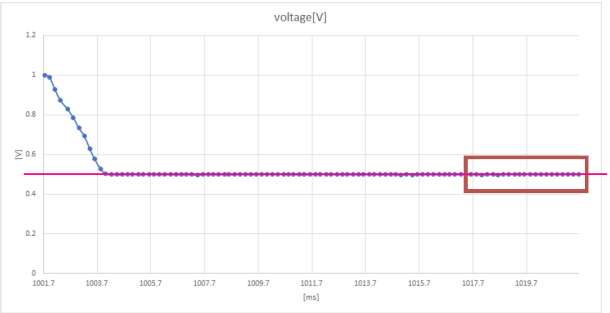
Figure 7 characterizes the voltage transition dynamics using the hardware PMBus control path at a 400 kHz PMBus clock. We define voltage transition latency as the elapsed time from issuing a voltage-update request at the PowerManager interface to the point at which the measured rail voltage reaches the target and remains within a stable region around it. For the 1.0 V → 0.5 V transition in Figure 7a, the measured end-to-end transition completes in 2.3 ms.

Figure 7b shows voltage trajectories for multiple target voltages. Under this hardware-PMBus 400 kHz configuration, the results show a monotonic relationship between the voltage step size $\Delta V$ and the transition time. Larger steps, such as 1.0 V → 0.5 V, take longer than smaller steps, such as 1.0 V → 0.9 V. This behavior motivates the later comparison of rising and falling transitions across control-path implementations and PMBus clock rates.

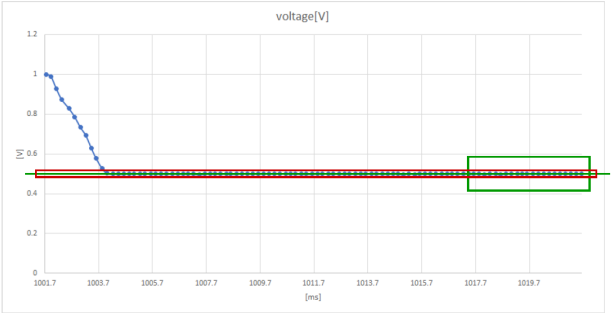
### C. Impact of Control Path and PMBus Clock Rate

Figure 8 compares voltage-transition measurements across four configurations: hardware-based PMBus at 400 kHz and 100 kHz, and software-based PMBus at 400 kHz and 100 kHz. The main effect of both the control path and the PMBus clock rate appears in the measurement interval during the transition, which is dominated by the time required to execute each PMBus transaction and collect a readback sample.

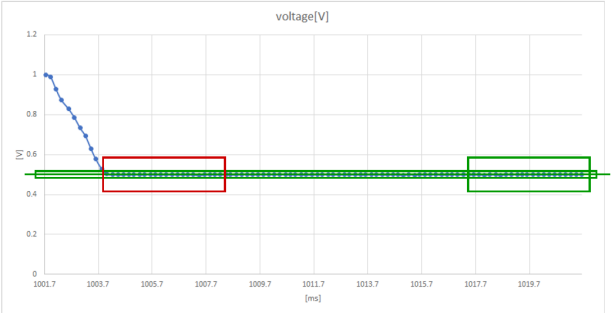
Table VI summarizes the approximate interval reported for each configuration. The hardware-based control path achieves



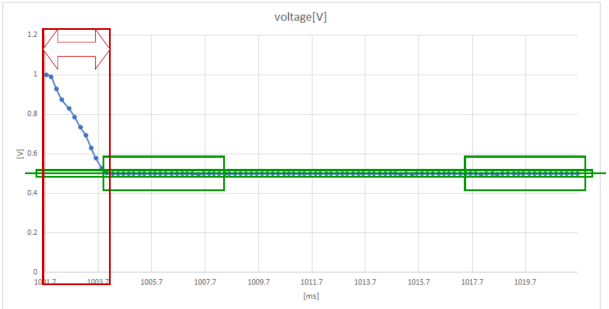

(a) Compute stable-voltage average $v_{\text{avg}}$ from the last $N$ samples




(b) Define the stability band $v_{\text{avg}} \pm x\%$




(c) Detect the first time $t_s$ with $N$ consecutive stable samples




(d) Settling time is the time from $t = 0$ to $t_s$

Fig. 9: Settling-time detection method.

TABLE VI: Approximate measurement interval per configuration.

| Configuration | Interval [ms] |
|---|---|
| HW-based PMBus, 400 kHz | 0.2 |
| HW-based PMBus, 100 kHz | 0.6 |
| SW-based PMBus, 400 kHz | 0.8 |
| SW-based PMBus, 100 kHz | 1.0 |

the finest measurement granularity, 0.2 ms at 400 kHz and 0.6 ms at 100 kHz. The software-based control path incurs substantially larger intervals, 0.8 ms at 400 kHz and 1.0 ms at 100 kHz, due to processor-side execution overhead in addition to the PMBus transaction time. These differences directly affect how quickly the controller can observe rail behavior and how densely it can sample the transient response, which later impacts settling-time detection and the repeatability of latency measurements.

### D. Settling-Time Detection Methodology

To report controller latency consistently across configurations, we compute a settling time from the sampled voltage readback trace, following the procedure summarized in Figure 9. The method operates on a sequence of voltage samples $v[0], v[1], \ldots, v[T]$ obtained during a transition. In Figure 9a, the stable voltage estimate is computed as the average of the last $N$ samples, $v_{\text{avg}} = \frac{1}{N}\sum_{k=T-N+1}^{T} v[k]$. Figure 9b then defines a stability band around this value as $v_{\text{avg}} \pm x\%$, and samples within this band are classified as stable. The next step, illustrated in Figure 9c, is to locate the first time index $t_s$ such that $N$ consecutive samples beginning at $t_s$ are stable. Finally, Figure 9d defines the settling time as the elapsed time from the first sample ($t = 0$) to $t_s$. This criterion makes the definition robust to transient overshoot and measurement noise, while remaining simple to reproduce across different PMBus clock rates and control-path implementations.

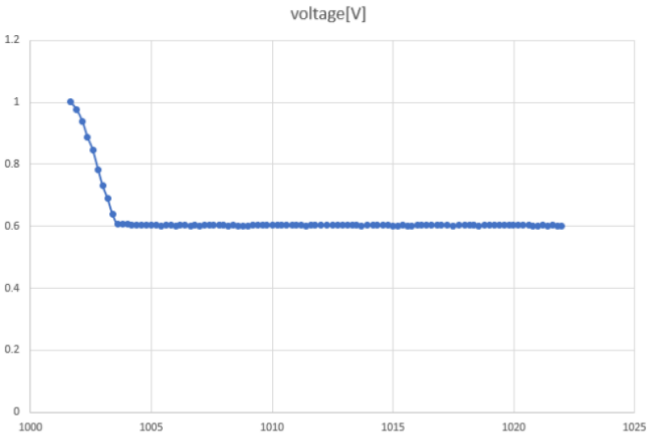
### E. Validation Against External Measurement

To validate the internal PMBus-based voltage readback measurements used throughout this section, we compare them against an external oscilloscope measurement of the same voltage transition. Figure 10a shows the voltage trace obtained from periodic PMBus readback using `READ_VOUT`, while Figure 10b shows the oscilloscope waveform captured at the same rail.

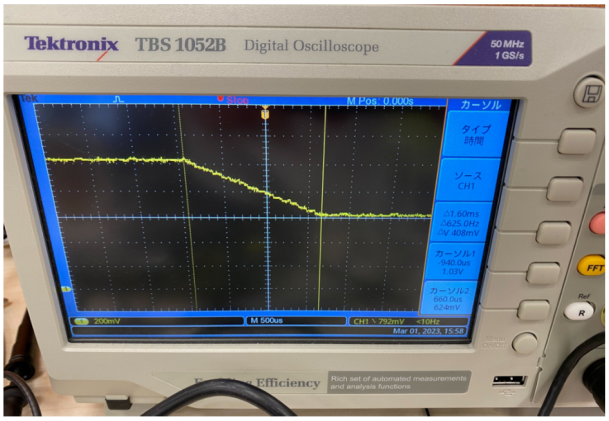
The two measurements reach the same target voltage and show the same overall transition trend. A visible timing offset can occur because the PMBus-based trace is derived from discrete samples taken at the measurement interval determined by the selected control path and PMBus clock rate, whereas the oscilloscope provides higher-time-resolution analog sampling. In the remainder of this section, we use the PMBus-based readback for consistency across configurations and apply the settling-time detection methodology described in Section V-D to infer transition latency from sampled observations.

### F. Controller Overhead

We quantify the overhead of integrating VolTune by reporting FPGA resource utilization and estimated static power




(a) PMBus-based voltage readback




(b) Oscilloscope waveform

Fig. 10: PMBus readback and oscilloscope validation.

TABLE VII: Hardware-based implementation resource utilization.

| Component/Resource | Slice LUTs (203800) | Slice Reg. (407600) | Slices (50950) | LUT as Logic (203800) | LUT as Mem. (64000) | BRAM Tile (445) | DSPs (840) |
|---|---|---|---|---|---|---|---|
| `Counter module` | <0.01% | 0.02% | 0.03% | <0.01% | 0.00% | 0.00% | 0.00% |
| `PowerManager module` | 0.31% | 0.46% | 1.19% | 0.31% | 0.02% | 0.00% | 0.24% |
| `PMBus module` | 0.12% | 0.03% | 0.15% | 0.12% | 0.00% | 0.00% | 0.00% |
| `Total` | 1.45% | 1.30% | 3.48% | 1.22% | 0.72% | 1.80% | 0.24% |

TABLE VIII: Software-based implementation resource utilization.

| Component/Resource | Slice LUTs (203800) | Slice Reg. (407600) | Slices (50950) | LUT as Logic (203800) | LUT as Mem. (64000) | BRAM Tile (445) | DSPs (840) |
|---|---|---|---|---|---|---|---|
| `axi_gpio` | 0.03% | 0.02% | 0.05% | 0.03% | 0.00% | 0.00% | 0.00% |
| `axi_timer` | 0.10% | 0.04% | 0.16% | 0.10% | 0.00% | 0.00% | 0.00% |
| `axi_uartlite` | 0.05% | 0.03% | 0.09% | 0.04% | 0.02% | 0.00% | 0.00% |
| `axis_dwidth_converter` | <0.01% | 0.06% | 0.11% | <0.01% | 0.00% | 0.00% | 0.00% |
| `mdm_1` | 0.05% | 0.03% | 0.08% | 0.04% | 0.01% | 0.00% | 0.00% |
| `microblaze` | 0.76% | 0.31% | 1.12% | 0.70% | 0.19% | 0.00% | 0.36% |
| `microblaze_local_memory` | 0.36% | 0.32% | 0.98% | 0.24% | 0.40% | 57.53% | 0.00% |
| `pmbus_io` | 0.00% | 0.00% | 0.00% | 0.00% | 0.00% | 0.00% | 0.00% |
| `smartconnect` | 0.19% | 0.09% | 0.36% | 0.19% | <0.01% | 0.00% | 0.00% |
| `util_vector_logic` | <0.01% | 0.00% | <0.01% | <0.01% | 0.00% | 0.00% | 0.00% |
| `Total` | 1.53% | 0.90% | 2.81% | 1.34% | 0.62% | 57.52% | 0.36% |

for the hardware-based and software-based implementations. Resource utilization is extracted from Vivado implementation reports, and static power is extracted from Vivado power reports. The hardware-based implementation realizes the PowerManager subsystem and PMBus transaction engine as dedicated FPGA logic, as shown in Figure 1, while the software-based implementation additionally instantiates a MicroBlaze processor subsystem together with its associated memory and interconnect, as shown in Figure 2.

***FPGA resource utilization:*** Table VII and Table VIII report the Vivado hierarchical utilization breakdown for the hardware-based and software-based VolTune implementations, respectively. Percentages are shown relative to the KC705 device totals listed in the column headers.

After excluding debug logic, the main overhead difference between the two implementations is not uniform across all resource classes. The software-based implementation remains close to the hardware-based implementation in Slice LUTs, 1.53% versus 1.45%, and increases DSP usage from 0.24% to 0.36% (1.50×). At the same time, its Slice Register and Slice utilization percentages are lower than those of the hardware-based implementation, 0.90% versus 1.30% for Slice Registers and 2.81% versus 3.48% for Slices. The dominant difference is Block RAM utilization, which increases from 1.80% to 57.52% (31.96×), indicating that the primary cost of the software control path is the processor memory subsystem rather than additional PMBus-specific logic.

The breakdown also shows where the overhead originates. In the software-based design, `microblaze_local_memory` alone consumes 57.53% of BRAM tiles, accounting for essentially all of the total BRAM usage. In contrast, the remaining infrastructure, including `smartconnect` (0.19% LUTs, 0.09% registers, 0.36% slices) and AXI peripherals such as `axi_timer` and `axi_uartlite`, contributes relatively small fractions. This indicates that the software implementation overhead

TABLE IX: VolTune static power breakdown.

| Implementation | Module | W | Share |
|---|---|---|---|
| Hardware | PowerManager module | 0.011 | 1% |
| | PMBus module | 0.003 | 1% |
| | Counter module | 0.001 | <1% |
| | **Total** | **0.015** | **2%** |
| Software | microblaze | 0.052 | 6% |
| | microblaze_local_memory | 0.023 | 3% |
| | smartconnect | 0.003 | 1% |
| | axi_timer | 0.002 | <1% |
| | axis_dwidth_converter | 0.001 | <1% |
| | axi_uartlite | 0.001 | <1% |
| | mdm_1 | 0.001 | <1% |
| | axi_gpio | 0.001 | <1% |
| | **Total** | **0.084** | **9%** |

is concentrated in instantiating a general-purpose execution substrate rather than in scaling the PMBus transaction engine itself.

For the hardware-based implementation, the PMBus-specific modules remain lightweight. The `PMBus module` consumes 0.12% Slice LUTs and 0.15% slices with no BRAM usage, while the `PowerManager module` consumes 0.31% Slice LUTs and 1.19% slices with no BRAM usage and 0.24% DSP usage. The `Counter module` is negligible across all resources. As a result, the hardware control path provides a substantially lower-memory integration point, and its overhead is dominated by standard FPGA logic resources rather than BRAM, which is often the limiting resource for accelerator designs.

Overall, these results support the design choice of providing both control paths. The hardware-based control path offers the lower-memory and lower-static-power implementation, while the software-based control path preserves comparable logic utilization but trades a 31.96× increase in BRAM utilization for programmability and ease of extension.

***Static power estimation:*** Table IX reports the Vivado static power breakdown for the hardware-based and software-based VolTune implementations. A direct comparison shows that the software-based control path increases static power from 0.015 W to 0.084 W (5.60×), reflecting the always-on processor and memory infrastructure required by the MicroBlaze-based execution substrate.

The breakdown indicates that the software-based overhead is dominated by the processor subsystem. The `microblaze` core (0.052 W) and `microblaze_local_memory` (0.023 W) contribute 0.075 W combined, accounting for 89.3% of the 0.084 W total. In contrast, the remaining AXI peripherals and interconnect blocks each contribute 0.003 W or less, indicating that the static-power overhead is concentrated in the general-purpose execution substrate rather than in peripheral glue logic.

For the hardware-based implementation, the dominant contributor is the `PowerManager module` at 0.011 W, while the remaining support modules each contribute 0.003 W or less. This confirms that the hardware control path has a very small always-on static-power footprint compared with the software-based alternative.

### G. Summary

Overall, the controller characterization shows that the hardware-based VolTune control path has a low integration cost in both power and FPGA resources. Its always-on static-power overhead remains around only a few percent of the subsystem budget, and its resource usage occupies only a small fraction of the KC705 device. These results indicate that VolTune can operate as a practical runtime control utility block without materially reducing the power or resource budget available to the main application logic.

## VI. Representative Case Study

### A. Case-Study Target and Rationale

We evaluate VolTune on the KC705 high-speed serial transceivers as a representative, voltage-sensitive subsystem. The case study targets the transceiver supply rails generated by the Texas Instruments UCD9248 power controller and therefore directly programmable through the PMBus-controlled regulator [29].

Serial transceivers are a practical target because voltage tuning can be evaluated using application-visible metrics under multi-gigabit operation, primarily bit error rate (BER) and also link latency. In addition, transceiver rails are largely decoupled from core logic supplies, which enables targeted rail-level experiments without modifying the FPGA bitstream or the timing closure of unrelated logic.

The results in this section are intended as a representative demonstration. The objective is to show that VolTune can be used to sweep rail voltages at runtime, quantify energy and reliability behavior, and identify a bounded operating region on a commercial FPGA platform.

### B. Experimental Setup and Methodology

This case study evaluates runtime voltage tuning on the KC705 transceiver subsystem through repeatable link tests. The evaluation loop programs the target rail voltage through VolTune and measures link correctness and performance at each operating point. Voltage updates and readback reuse the same VolTune opcode and PMBus control mechanisms described in Section IV.

***Board setup:*** Figure 11 shows the board-level setup used for the transceiver case study. The evaluation uses two KC705 boards connected back-to-back through the on-board high-speed transceiver links. A Windows Host PC runs the test program and controls both boards through JTAG using `hw_server` and XSDB. A dedicated external clock generator, the Skyworks Si5391A-A evaluation board [30], provides the reference clocks used by the transceivers on both KC705 boards. In our setup, the clock generator outputs 125.000 MHz for 2.5, 5.0, and 10.0 Gbps tests, and 117.188 MHz for 7.5 Gbps tests, as summarized in Table X. The selected reference clock is routed to the corresponding transceiver reference clock inputs of both boards, ensuring that both ends of the link operate from a consistent and repeatable reference clock during each voltage sweep.

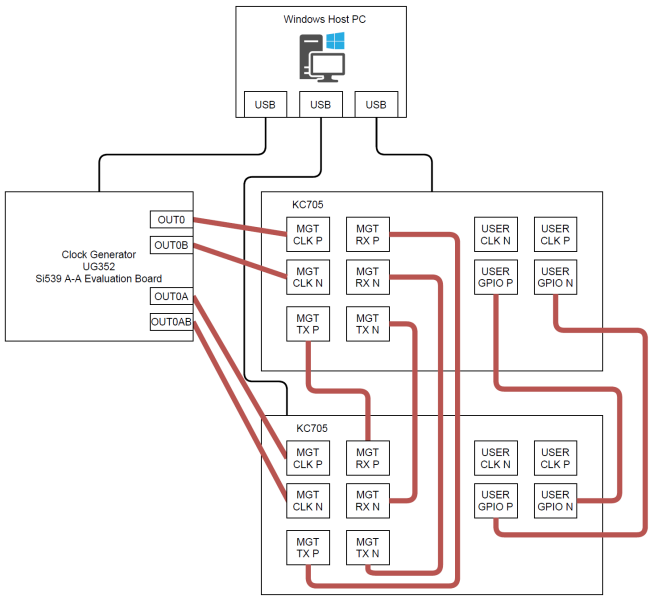



Fig. 11: Board setup for the transceiver case study.

TABLE X: Case-study test conditions.

| Item | Value |
|---|---|
| Target rails | `MGTAVCC` swept<br>`MGTAVTT` fixed at nominal |
| Voltage sweep | Range: 1.0 V → 0.7 V<br>Step: 0.001 V |
| Sweep modes | TX and RX swept together<br>TX fixed at 1.0 V, RX swept<br>RX fixed at 1.0 V, TX swept |
| Link speed (reference clock) | 2.5 Gbps (125.000 MHz)<br>5.0 Gbps (125.000 MHz)<br>7.5 Gbps (117.188 MHz)<br>10.0 Gbps (125.000 MHz) |
| Test payload | 10 GByte count-up data stream |
| PMBus control path | Hardware-based PMBus |
| PMBus clock rate | 400 kHz |
| Reported metrics | Received data size<br>Bit error rate (BER)<br>Latency<br>Rail power |

***Target voltage rail:*** We evaluate VolTune in a representative setting based on the GTX transceivers on KC705 [31]. In this case study, voltage tuning is applied to the GTX transceiver power domain by sweeping the analog supply rail `MGTAVCC`, previously listed in Table II. For Kintex-7 devices, `MGTAVCC` supplies the GTX transmitter and receiver analog circuits, while `MGTAVTT` supplies the termination circuitry [31], [32]. In our setup, `MGTAVTT` is held at its nominal configuration while `MGTAVCC` is swept, to isolate the effect of analog supply scaling.

***Workload and sweep procedure:*** For each voltage point, the transmitter sends a 10 GByte count-up data stream and the receiver checks correctness to measure received data size and bit error rate (BER). Tests are repeated across link speeds of 2.5, 5.0, 7.5, and 10.0 Gbps. To separate transmitter-side and receiver-side sensitivity, we sweep the transceiver supply rail for the TX side and the RX side independently using three modes: (i) TX and RX swept together, (ii) TX fixed at nominal 1.0 V with RX swept, and (iii) RX fixed at nominal 1.0 V with TX swept. Table X summarizes the full set of test conditions used in this case study.

### *C. Bit Error Rate and Throughput Under Voltage Tuning*

We first evaluate how link reliability changes when the transceiver analog supply rail is reduced at runtime. We use the 10.0 Gbps link setting and sweep the target rail from 1.0 V down to 0.7 V using the procedure described in Section VI-B. For each voltage point, the transmitter sends a 10 GByte count-up stream and the receiver reports both the received data size and the bit error rate (BER). Received data size is included to distinguish gradual BER degradation from hard link failure or loss of synchronization.

Figure 12 reveals three regimes that are relevant to runtime voltage selection. First, the link delivers the full payload and BER remains effectively zero down to approximately 0.869 V, as shown by the flat region in Figure 12b. Second, errors begin to appear at approximately 0.869 V, and BER increases from the $10^{-10}$–$10^{-9}$ range near 0.869–0.868 V to approximately $10^{-7}$ near 0.866 V, and to approximately $10^{-6}$ near 0.864 V in Figure 12c. Third, at lower voltages the link becomes unstable and the received data size drops sharply, with the first major throughput collapse occurring near 0.80 V in Figure 12a.

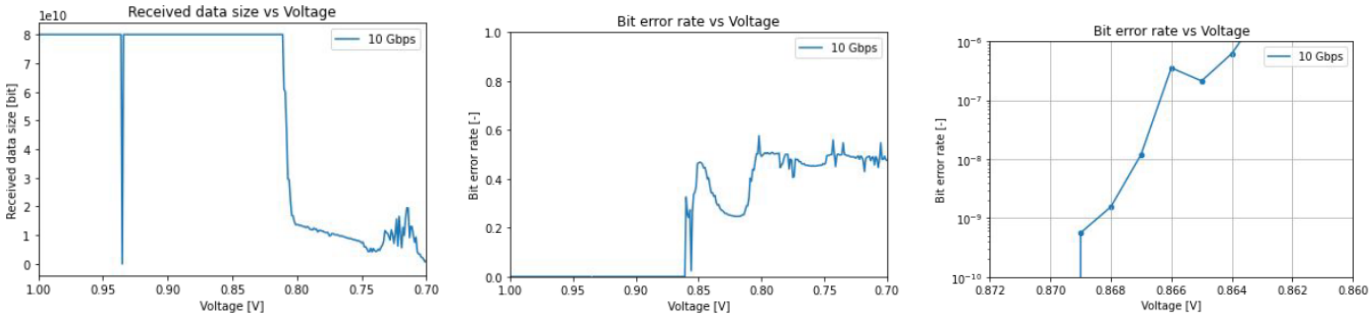
These observations indicate that runtime tuning can operate within a bounded usable region above the collapse point, and that the voltage range near 0.869–0.864 V forms a transition region where BER increases by several orders of magnitude without an immediate loss of link activity. In the following subsections, we use these regimes as a reference point to separate TX-side and RX-side sensitivity and to relate error onset to latency and rail power.

### *D. Sensitivity Study, TX-Only vs RX-Only Voltage Scaling*

We next separate TX-side and RX-side voltage sensitivity by comparing three sweep modes: (i) TX and RX swept together, (ii) TX swept while RX is fixed at 1.0 V, and (iii) RX swept while TX is fixed at 1.0 V. Figure 13 summarizes the received data size and BER trends for the three modes at 10.0 Gbps.

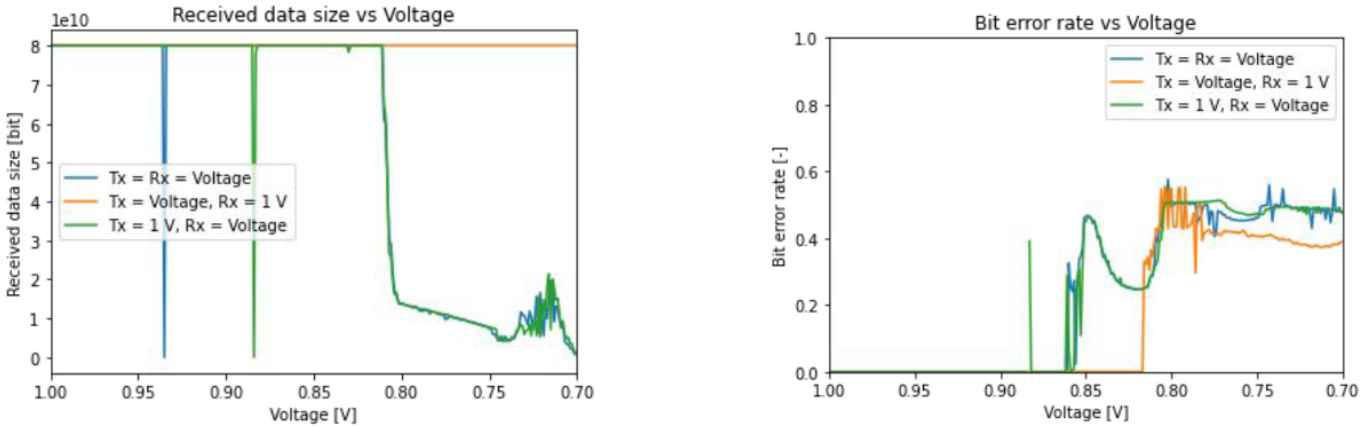
Two quantitative differences stand out. First, when RX is fixed at 1.0 V and only TX voltage is swept, the received data size remains at the full payload down to 0.7 V. In contrast, when RX voltage is swept (either TX=RX swept together or TX fixed at 1.0 V), the received data size begins to decrease at approximately 0.81 V (Figure 13a). Second, the BER onset voltage shifts substantially depending on whether RX is swept. When RX is fixed, BER does not increase until approximately 0.82 V, while in the RX-swept cases BER begins to increase at approximately 0.87 V. These onset voltages are derived from the corresponding zoomed BER analysis for the same measurements, which is omitted here for brevity.

Overall, these results indicate that RX-side voltage reduction has a larger impact on both throughput stability and BER




(a) Received data size (b) BER over full sweep (c) BER near the onset region (log scale)

Fig. 12: 10.0 Gbps reliability under voltage tuning.




(a) Received data size (b) BER (full sweep)

Fig. 13: TX-only vs RX-only voltage scaling at 10.0 Gbps.

than TX-side voltage reduction in this setup. In the following subsections, we use this asymmetry to interpret how speed and power trends evolve under voltage tuning.

### *E. Impact of Link Speed*

We next evaluate how the voltage sensitivity shifts with transceiver link speed. Figure 14 compares received data size and BER over the full voltage sweep for 2.5, 5.0, 7.5, and 10.0 Gbps under the same test methodology in Section VI-B.

Figure 14 shows that higher link speed requires higher voltage to maintain low-error operation. For 10.0 Gbps, BER begins to rise at approximately 0.869 V. For 7.5 Gbps, the BER onset shifts down to approximately 0.787 V. For 5.0 Gbps and 2.5 Gbps, the BER onset occurs at approximately 0.745 V and 0.744 V, respectively. These onset voltages indicate a widening voltage headroom as link speed decreases, which increases the available tuning range for reducing rail power.

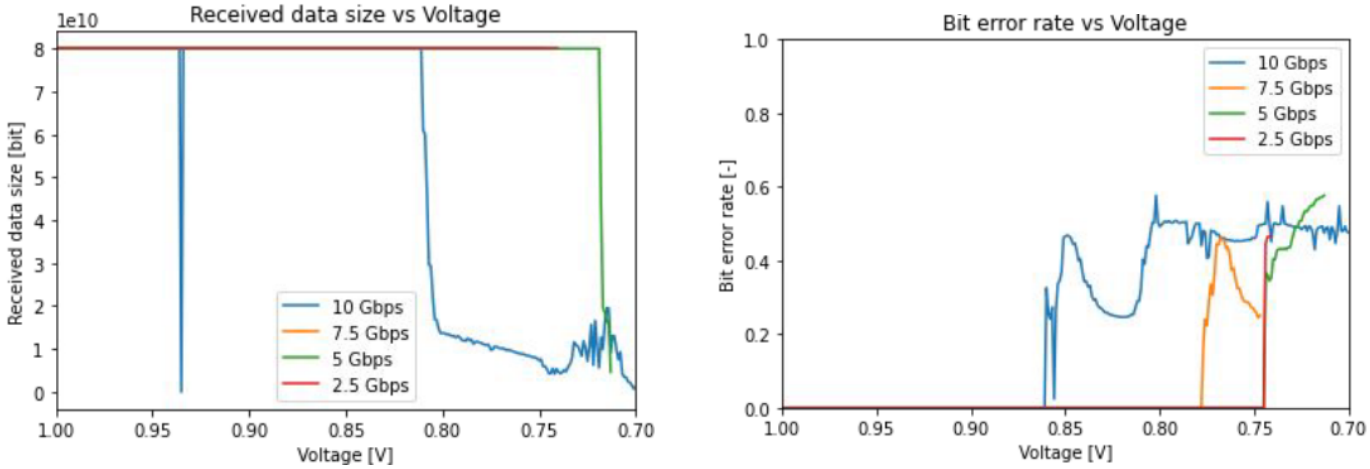
Received data size provides a complementary view of stability. In Figure 14a, the 10.0 Gbps case exhibits the earliest throughput collapse, with a major drop in received data size near 0.80 V. In contrast, the 5.0 Gbps case maintains full received data size to a lower voltage and then drops near 0.72 V. The 7.5 Gbps and 2.5 Gbps tests terminate before a clear received-data-size collapse is observed in the plotted sweep, so BER onset provides the primary stability indicator for these two speeds in this dataset.

Overall, these results show that the usable voltage range depends strongly on link speed. The transition region shifts downward as speed decreases, enabling more aggressive voltage reduction at lower rates while preserving bounded BER and stable throughput.

### *F. Latency Impact*

We next evaluate how voltage tuning impacts end-to-end link latency. Figure 15 reports measured latency as a function of swept voltage and link speed. We present two complementary views: a comparison across TX-only versus RX-only scaling modes (Figure 15a) and a comparison across link speeds (Figure 15b).

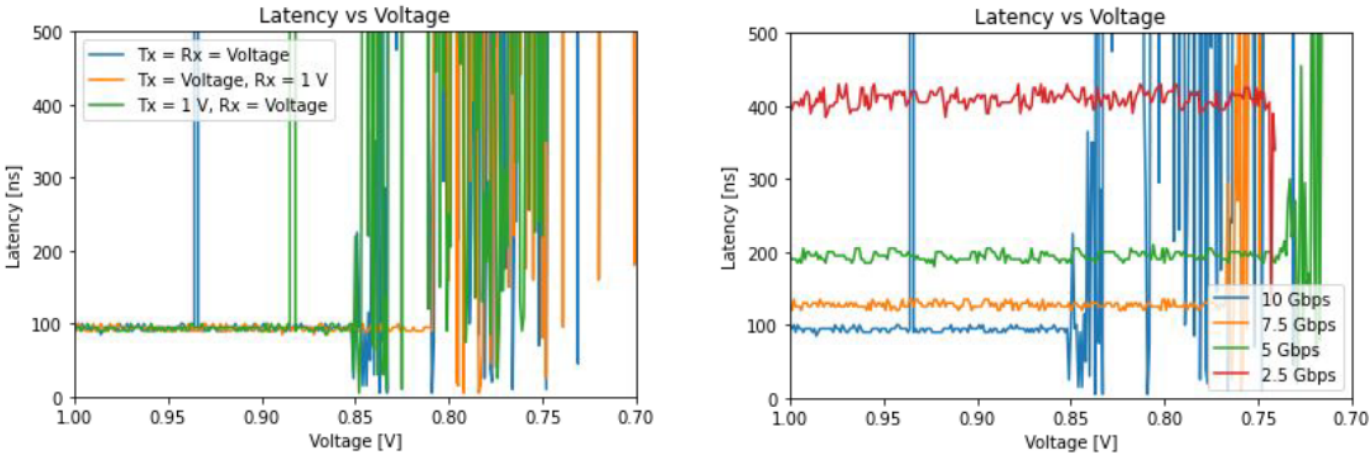
In the mode comparison (Figure 15a), latency remains approximately constant at about 90–100 ns over the high-voltage region, and then exhibits large excursions as voltage is reduced further. The onset of sustained latency excursions appears at approximately 0.86 V, after which all three sweep modes exhibit frequent spikes. This aligns with the reliability transition observed in Figure 12, where BER begins rising near 0.869 V, and with the mode dependence in Figure 13, where RX-swept configurations exhibit earlier degradation than TX-only scaling.




(a) Received data size

(b) BER (full sweep)

Fig. 14: Link-speed impact under voltage tuning.




(a) TX-only vs RX-only scaling modes

(b) Link-speed comparison

Fig. 15: Latency under voltage tuning.

Latency depends strongly on link speed in the stable region (Figure 15b). In the nominal operating region, the measured latency is approximately 100 ns at 10.0 Gbps, 130 ns at 7.5 Gbps, 200 ns at 5.0 Gbps, and 410 ns at 2.5 Gbps. These baseline values remain approximately constant over the voltage range where each curve is stable. As voltage is reduced further, latency becomes disturbed with large excursions, starting at approximately 0.86 V for 10.0 Gbps, approximately 0.76 V for 7.5 Gbps, and approximately 0.74–0.75 V for 5.0 Gbps. This ordering matches the speed-dependent reliability headroom seen in Figure 14, where higher speeds show earlier BER onset and a narrower usable voltage range.

### G. Power Reduction

We next quantify the power reduction achieved by voltage tuning on the transceiver supply rails. The goal is to separate three effects: how TX-side and RX-side power respond to rail scaling, how link speed shifts the baseline power, and how much power can be saved while remaining in the near-zero BER region, that is, before the BER onset around 0.869 V observed in Figure 12.

***TX vs RX power trends:*** The TX-side and RX-side power traces show a clear directional dependence on which side is being voltage-scaled. When the TX-side rail is swept, TX power decreases monotonically with voltage, while keeping the TX rail fixed keeps TX power approximately constant. The same pattern holds for RX power when the RX-side rail is swept. Table XI summarizes the observed trends and representative power levels extracted from the measured power traces.

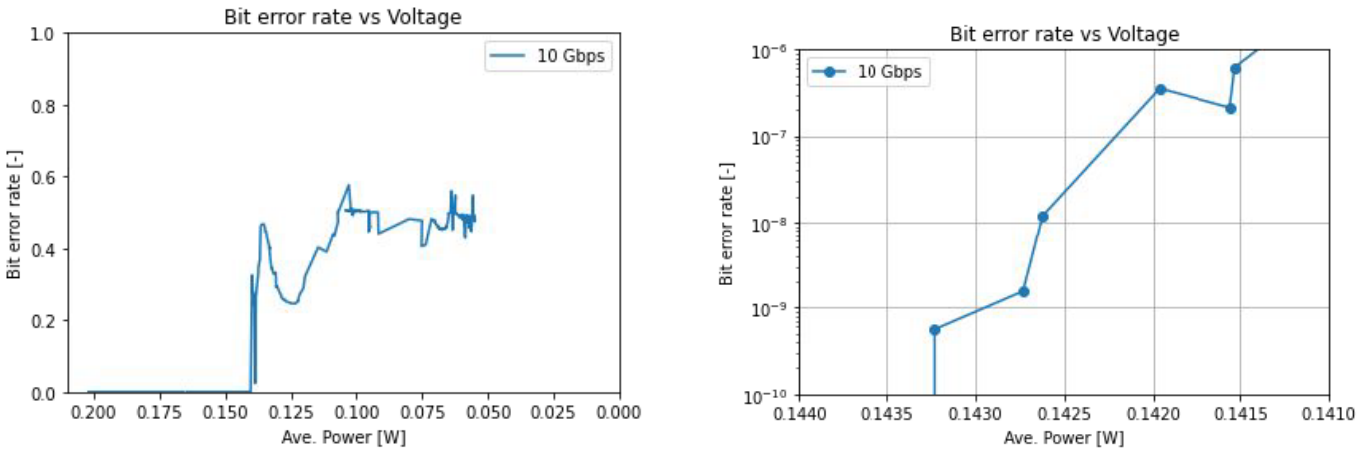
Two practical observations follow. First, the savings from voltage tuning are largely localized to the side whose rail is being scaled, which explains why RX-side scaling has the strongest impact on reliability in Figure 13. Second, the power reduction is smooth with voltage, and there is no abrupt power discontinuity at the reliability transition. Selecting an operating point is therefore fundamentally a reliability-constrained optimization rather than a power-discontinuity problem.

TABLE XI: TX-side and RX-side power trends under different sweep modes.

| Sweep mode | TX-side power trend | RX-side power trend |
|---|---|---|
| TX = RX swept | Decreases from ∼0.20 W at 1.0 V to ∼0.08 W at 0.7 V | Decreases from ∼0.17 W at 1.0 V to ∼0.07–0.08 W at 0.7 V |
| TX swept, RX fixed at 1.0 V | Decreases from ∼0.20 W at 1.0 V to ∼0.08 W at 0.7 V | Approximately constant near ∼0.16–0.17 W |
| RX swept, TX fixed at 1.0 V | Approximately constant near ∼0.20 W | Decreases from ∼0.17 W at 1.0 V to ∼0.07–0.08 W at 0.7 V |

TABLE XII: Representative rail power versus link speed.

| Speed | TX power @ 1.0 V | RX power @ 1.0 V | TX power @ 0.8 V | RX power @ 0.8 V |
|---|---|---|---|---|
| 10.0 Gbps | ∼0.20 W | ∼0.17 W | ∼0.13 W | ∼0.11 W |
| 7.5 Gbps | ∼0.18 W | ∼0.15–0.16 W | ∼0.12 W | ∼0.10 W |
| 5.0 Gbps | ∼0.14 W | ∼0.12 W | ∼0.09 W | ∼0.08 W |
| 2.5 Gbps | ∼0.12 W | ∼0.09–0.10 W | ∼0.08 W | ∼0.07 W |




(a) BER vs power

(b) BER onset region (close-up)

Fig. 16: BER versus rail power at 10.0 Gbps.

***Link-speed power trend:*** Link speed shifts the baseline rail power in the stable region, and all curves drop as voltage is reduced. Table XII summarizes representative rail power values at 1.0 V and 0.8 V across link speeds, and we use these values to quantify both baseline differences and undervolting-induced savings.

At 1.0 V, increasing link speed from 2.5 Gbps to 10.0 Gbps raises the baseline rail power by approximately 66–70% on both TX and RX. This confirms that link speed influences not only the reliability headroom (Figure 14), but also the baseline operating point from which voltage tuning begins.

When undervolting from 1.0 V to 0.8 V, the fractional power reduction is largely similar across link speeds. On the TX side, the reduction is approximately 33–36% across all speeds. On the RX side, the reduction is also approximately 33–35% for most speeds, and becomes smaller at 2.5 Gbps at approximately 25–30%. This indicates that voltage scaling provides a comparable percentage power gain across speeds, and link speed primarily changes the baseline power level and the available voltage headroom before instability, rather than changing the efficiency of the voltage-to-power scaling itself.

This ordering matches the reliability headroom observed in Figure 14. Higher speeds not only show earlier BER onset, but they also operate from a higher baseline rail power. As a result, lowering the link speed offers two levers simultaneously, lower baseline rail power and a wider voltage headroom before entering the instability region.

***BER-aware power savings at 10.0 Gbps:*** Figure 16 quantifies the trade-off between link reliability and rail power at 10.0 Gbps by plotting BER as a function of measured average rail power. The full plot shows a long near-zero-BER plateau at higher power, followed by a rapid BER increase as power is reduced. The close-up isolates this onset region and allows power savings to be quantified at specific BER levels.

First, the power saving achievable *before the near-zero-BER boundary* is large. BER stays effectively zero down to approximately 0.1432 W in Figure 16b. Taking the 1.0 V operating point at approximately 0.20 W as the nominal baseline in the same plot, this corresponds to approximately 28.4% power reduction while remaining in the near-zero-BER regime.

Second, the additional power saving available in a *bounded, application-tolerable BER region* is comparatively small. In the close-up, BER rises from the $10^{-10}$–$10^{-9}$ range near 0.143–0.1425 W to approximately $10^{-7}$ near 0.1420 W and to approximately $10^{-6}$ near 0.1415 W. Allowing BER to increase from near-zero to approximately $10^{-6}$ therefore increases the cumulative power reduction from approximately 28.4% to approximately 29.3%, and the incremental gain across the

onset region itself is approximately 1.2% relative to the near-zero-BER boundary.

Third, larger power reductions are attainable only by entering the unstable regime. Below the onset region, the full BER-vs-power plot indicates that BER grows rapidly into the high-error range as power continues to drop. This behavior is consistent with the throughput collapse observed near 0.80 V in Figure 12 and with the latency-excursion regime in Figure 15, where the link becomes unstable rather than operating with bounded errors.

Overall, Figure 16 shows a clear structure: most of the practical power saving comes from tuning down to the near-zero-BER boundary, approximately 28% in this setup, while permitting bounded BER in the $10^{-7}$–$10^{-6}$ range provides only a small additional reduction before the system transitions into instability.

## VII. Discussion

### A. Architectural Role of VolTune

VolTune makes board-level voltage regulation a runtime-controlled architectural variable that can be invoked from within an FPGA system through a structured command interface. The PMBus control engine in Section IV provides a concrete, reproducible mapping from VolTune opcodes to PMBus commands and transaction sequences on KC705. The controller characterization in Section V shows that the control plane can, in its hardware realization, be kept always-on with low overhead, while still supporting millisecond-scale voltage transitions (Figure 7). The representative case study in Section VI further shows that runtime tuning exposes an interpretable energy, reliability trade-off with measurable boundaries using BER, received data size, latency, and rail power (Figures 12–16).

### B. Control Mechanism and Policy Separation

VolTune is designed as a control mechanism rather than as a fixed automatic optimizer. This is a deliberate architectural choice, because voltage-selection policy is inherently application- and platform-dependent. The present work therefore focuses on the instrumentation and command path needed to sweep voltages at runtime and to identify bounded operating regions under explicit constraints, for example near-zero BER versus bounded BER versus instability. This organization also makes the design easier to customize. Users can attach workload-specific optimization, reliability-aware control, or safety-constrained operating-point selection on top of the existing control substrate without redesigning the PMBus control engine itself. Any deployment that changes voltage during mission execution will still require a system-level policy layer that defines constraints and selects operating points based on application requirements and safety margins.

### C. Timing and Measurement Boundaries

The achievable reaction speed is fundamentally bounded by external power-delivery dynamics and the PMBus transaction and sampling cadence. Even with a hardware control path, voltage transitions remain in the millisecond regime and measurements are derived from discrete readback samples rather than continuous analog observation (Section V). This is sufficient for phase-level tuning and for operating-point exploration, but it is not intended for fast droop compensation or cycle-scale control. Relatedly, the settling-time definition depends on the sampling interval and stability parameters in Section V-D, which should be reported and tuned consistently when comparing platforms. At the same time, the modular structure of the control plane makes such implementation choices explicit, which helps users adapt the measurement and control methodology to their own timing requirements.

### D. Platform Dependence and Portability

VolTune's platform dependence is primarily a matter of mapping rather than architecture. The PMBus transaction layer is general, but each board requires a rail map and regulator configuration, as illustrated by the KC705 lane-to-(address,`PAGE`) mapping in Table II. Porting to another PMBus-controlled FPGA board is therefore straightforward at the architectural level but still requires practical board-specific work, including identifying programmable rails, verifying safe voltage bounds, and validating measurement interpretation against the chosen regulator's behavior. This is also an area where the design is intentionally extensible. Users can adapt the rail map, add board-specific PMBus commands, and refine the transaction layer without changing the overall architectural organization.

### E. Case-Study Scope and Generality

The case study also has clear scope boundaries. The transceiver rail is a strong representative target because it provides clear external metrics, but the quantitative boundaries reported here reflect a specific board, regulator configuration, link setup, and workload (Table X). The RX-dominant sensitivity observed in Figure 13 should be treated as an empirical result for this setup, not a universal statement about all transceiver deployments. In addition, this paper does not attempt to demonstrate multi-rail coordination or interactions between rails, and it does not model second-order effects such as rail coupling, regulator cross-effects, or thermal transients during long runs.

These scope limits do not reduce the architectural relevance of the design. Rather, they clarify the role of the present evaluation, which is to validate that VolTune can expose runtime voltage control as a practical mechanism and make application-visible trade-offs measurable on a commercial FPGA platform. The same control structure can be reused for other PMBus-controlled rails and other workloads, provided that the corresponding operating boundaries are characterized for the target setting.

### F. Hardware-Software Trade-offs

The two control paths provide different but complementary integration points. The hardware control path is the better fit when the control plane must remain lightweight, deterministic, and continuously available with minimal resource cost.

The software control path is useful for accessibility, rapid experimentation, and feature extension, because it simplifies changes to control logic and sequencing behavior. However, this flexibility can be expensive in FPGA memory resources and always-on static overhead (Section V-F). This trade-off is acceptable when BRAM is not a limiting resource and when rapid customization is needed, but it is a poor fit for BRAM-constrained accelerator designs where the control plane must remain a minimal add-on.

### G. Takeaway

Overall, VolTune is best interpreted as a reusable and customizable runtime voltage-control utility block. It provides a concrete, implementable PMBus control engine and an FPGA-invocable abstraction for rail tuning, while intentionally separating low-level voltage actuation from higher-level policy decisions. This separation makes the design practical to adapt. Users who require different PMBus commands, additional telemetry, alternative settling criteria, or platform-specific control policies can extend the implementation without changing the core architectural structure. To support reproducibility and follow-on use, the implementation is released as an open-source repository containing the design artifacts used in this work [14]. Policy design, safety envelopes, and multi-rail orchestration remain explicit future work.

## VIII. Conclusion

This paper presented VolTune, an open-source fine-grained runtime voltage control architecture that enables runtime tuning of FPGA supply rails through an FPGA-integrated control plane that abstracts PMBus transactions. VolTune provides both a hardware control path and a software control path, allowing designers to select between deterministic low-latency sequencing and processor-driven flexibility depending on system constraints.

On the KC705 platform with a TI UCD9248 regulator, the hardware control path completes an end-to-end voltage transition, including regulator settling, in 2.3 ms. The hardware-control-path overhead is small. The hardware implementation occupies 1.45% Slice LUTs and 1.80% BRAM tiles, and, excluding debug logic, consumes 0.015 W of static power, corresponding to 2% of the KC705 static power budget reported by Vivado for this subsystem.

A representative case study on GTX transceivers showed that runtime voltage tuning exposes bounded operating regions that can be identified using BER, received data size, latency, and rail power. At 10.0 Gbps, BER remains effectively zero down to approximately 0.869 V, then increases rapidly in the 0.869–0.864 V region, and throughput collapses near 0.80 V. RX-side voltage scaling dominates degradation in this setup, and the usable voltage headroom widens as link speed decreases, with BER onset shifting downward at lower rates. In terms of power, most practical savings are achieved before the near-zero-BER boundary, approximately 28.4% in the 10.0 Gbps setting, while permitting BER up to $10^{-6}$ yields only a small additional reduction, approximately 29.3% total, before instability.

Future work includes integrating closed-loop voltage-selection policies on top of the existing control plane, extending the implementation to additional rails and telemetry channels, and evaluating multi-rail coordination under workload-driven tuning scenarios.

## References


[1] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.

[2] M. Satyanarayanan, "The emergence of edge computing," *Computer*, vol. 50, no. 1, pp. 30–39, 2017.

[3] S. Mittal, "A survey of fpga-based accelerators for convolutional neural networks," *Neural Computing and Applications*, vol. 32, pp. 1109–1139, 2020.

[4] A. Putnam, A. M. Caulfield, E. S. Chung, D. Chiou, K. Constantinides, J. Demme, H. Esmaeilzadeh, J. Fowers, G. P. Gopal, J. Gray, M. Haselman, S. Hauck, S. Heil, A. Hormati, J.-Y. Kim, S. Lanka, J. Larus, E. Peterson, S. Pope, A. Smith, J. Thong, P. Y. Xiao, and D. Burger, "A reconfigurable fabric for accelerating large-scale datacenter services," *IEEE Micro*, vol. 35, no. 3, pp. 10–22, 2015.

[5] D. Firestone, A. Putnam, S. Mundkur, D. Chiou, A. Dabagh, M. Andrewartha, H. Angepat, V. Bhanu, A. Caulfield, E. Chung, H. K. Chandrappa, S. Chaturmohta, M. Humphrey, J. Lavier, N. Lam, F. Liu, K. Ovtcharov, J. Padhye, G. Popuri, S. Raindel, T. Sapre, M. Shaw, G. Silva, M. Sivakumar, N. Srivastava, A. Verma, Q. Zuhair, D. Bansal, D. Burger, K. Vaid, D. A. Maltz, and A. Greenberg, "Azure accelerated networking: SmartNICs in the public cloud," in *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*. Renton, WA: USENIX Association, Apr. 2018, pp. 51–66. [Online]. Available: https://www.usenix.org/conference/nsdi18/presentation/firestone

[6] Amazon Web Services, "The security design of the aws nitro system," AWS Whitepaper, 2023. [Online]. Available: https://docs.aws.amazon.com/whitepapers/latest/security-design-of-aws-nitro-system/security-design-of-aws-nitro-system.html

[7] Advanced Micro Devices, Inc., "Kintex ultrascale fpgas data sheet: Dc and ac switching characteristics," Datasheet DS892, 2025, dS892 (v1.20), August 20, 2025. [Online]. Available: https://docs.amd.com/v/u/en-US/ds892-kintex-ultrascale-data-sheet

[8] Intel Corporation, "Intel stratix 10 device datasheet," Datasheet, 2023, document date: 2023-12-08. [Online]. Available: https://www.intel.com/programmable/technical-pdfs/683181.pdf

[9] L. A. Barroso, J. Clidaras, and U. Hölzle, *The Datacenter as a Computer: An Introduction to the Design of Warehouse-Scale Machines*. Morgan & Claypool Publishers, 2013.

[10] A. Salami, M. Shafique, M. Ali, O. Unsal, and A. Cristal, "Comprehensive evaluation of supply voltage underscaling in fpga on-chip memories," in *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2018, pp. 724–736.

[11] J. Han and M. Orshansky, "Approximate computing: An emerging paradigm for energy-efficient design," *Computer*, vol. 46, no. 2, pp. 58–65, 2013.

[12] S. Mittal, "A survey of techniques for approximate computing," *ACM Computing Surveys*, vol. 48, no. 4, pp. 62:1–62:33, 2016.

[13] System Management Interface Forum, "Pmbus power system management protocol specification," Specification archive, see PMBus specification archives. [Online]. Available: https://pmbus.org/specification-archives/

[14] A. Ben Ahmed, T. Hirofuchi, and T. Fukai, "Voltune: Open-source runtime voltage control architecture for fpga systems," https://github.com/CCIRT/VolTune, 2026, gitHub repository, accessed: 2026-03-17.

[15] C. T. Chow, L. S. M. Tsui, P. H. W. Leong, W. Luk, and S. J. E. Wilton, "Dynamic voltage scaling for commercial fpgas," in *Proceedings of the 2005 IEEE International Conference on Field-Programmable Technology (FPT)*, 2005, pp. 173–180.

[16] J. M. Levine, E. A. Stott, and P. Y. K. Cheung, "Dynamic voltage & frequency scaling with online slack measurement," in *Proceedings of the 2014 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, 2014, pp. 65–74.

[17] A. Farhadi Beldachi and J. Nunez-Yanez, "Accurate power control and monitoring in ZYNQ boards," in *2014 24th International Conference on Field Programmable Logic and Applications (FPL)*, 2014.

[18] S. Zhao, I. Ahmed, C. Lamoureux, A. Lotfi, V. Betz, and O. Trescases, "A universal self-calibrating dynamic voltage and frequency scaling (DVFS) scheme with thermal compensation for energy savings in FPGAs," in *2016 IEEE Applied Power Electronics Conference and Exposition (APEC)*, 2016, pp. 1882–1887.

[19] D. Ernst, N. S. Kim, S. Das, S. Pant, R. Rao, T. Pham, C. Ziesler, D. Blaauw, T. Austin, T. Mudge, and K. Flautner, "Razor: A low-power pipeline based on circuit-level timing speculation," in *Proceedings of the 36th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2003, pp. 7–18.

[20] S. Das, C. Tokunaga, S. Pant, W.-H. Ma, S. Kalaiselvan, K. Lai, D. Bull, and D. Blaauw, "Razor II: In situ error detection and correction for PVT and SER tolerance," *IEEE Journal of Solid-State Circuits*, vol. 44, no. 1, pp. 32–48, 2009.

[21] B. Salami, O. S. Unsal, and A. Cristal, "Evaluating built-in ecc of fpga on-chip memories for the mitigation of undervolting faults," in *2019 27th Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP)*, 2019, pp. 242–246.

[22] B. Salami, E. B. Onural, I. E. Yuksel, F. Koc, O. Ergin, A. Cristal, O. S. Unsal, H. Sarbazi-Azad, and O. Mutlu, "An experimental study of reduced-voltage operation in modern fpgas for neural network acceleration," in *2020 50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2020, pp. 138–149.

[23] A. Ben Ahmed, R. Takano, and T. Hirofuchi, "Exploring the potential of error-permissive communication in multi-fpga-based edge computing," in *2022 IEEE 15th International Symposium on Embedded Multicore/Many-core Systems-on-Chip (MCSoC)*, 2022, pp. 93–97.

[24] S. Ullah, S. Rehman, B. S. Prabakaran, F. Kriebel, M. A. Hanif, M. Shafique, and A. Kumar, "Area-optimized low-latency approximate multipliers for fpga-based hardware accelerators," in *Proceedings of the 55th Annual Design Automation Conference*. Association for Computing Machinery, 2018.

[25] S. Perri, F. Spagnolo, F. Frustaci, and P. Corsonello, "Efficient approximate adders for fpga-based data-paths," *Electronics*, vol. 9, no. 9, p. 1529, 2020.

[26] G. Korol, M. G. Jordan, M. B. Rutzig, and A. C. S. Beck, "Confax: Exploiting approximate computing for configurable fpga cnn acceleration at the edge," in *2022 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2022, pp. 1650–1654.

[27] Texas Instruments, "Ucd9248 digital pwm system controller," Datasheet, 2012, sLVSA33A, January 2010, Revised August 2012. [Online]. Available: https://www.ti.com/lit/ds/symlink/ucd9248.pdf

[28] System Management Interface Forum, "Pmbus power system management protocol, part ii, command language, version 1.1," Specification, 2007, revision 1.1, February 2007. [Online]. Available: https://pmbusprod.wpenginepowered.com/wp-content/uploads/2021/05/PMBus-Specification-Rev-1-1-Part-II-20070205.pdf

[29] Xilinx Inc., "Kc705 evaluation board for the kintex-7 fpga user guide," User Guide UG810, 2019, revision 1.9, Release Date 2019-02-04. [Online]. Available: https://docs.amd.com/v/u/en-US/ug810_KC705_Eval_Bd

[30] Skyworks Solutions, Inc., "Ug352: Si5391a-a evaluation board user's guide," User Guide, 2022, rev. 0.2. [Online]. Available: https://www.skyworksinc.com/-/media/Skyworks/SL/documents/public/user-guides/ug352-si5391a-a-evb.pdf

[31] Advanced Micro Devices, Inc., "7 series fpgas gtx/gth transceivers user guide (ug476)," User Guide, 2018, uG476 v1.12.1, Release Date 2018-08-14. [Online]. Available: https://docs.amd.com/v/u/en-US/ug476_7Series_Transceivers

[32] ——, "Kintex-7 fpgas data sheet: Dc and ac switching characteristics (ds182)," Product Specification, 2021, dS182 v2.19, Release Date 2021-03-26. [Online]. Available: https://docs.amd.com/v/u/en-US/ds182_Kintex_7_Data_Sheet